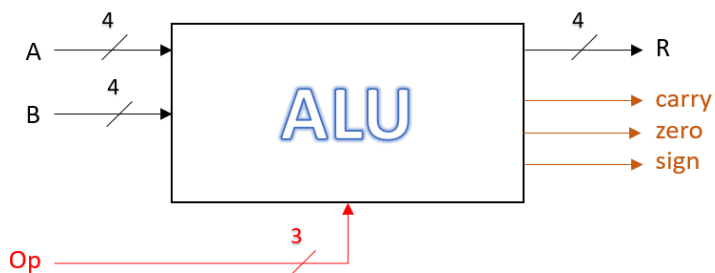


PRÁCTICA 1

El objetivo de la práctica 1 es servir de introducción a Verilog con un ejemplo de diseño puramente combinacional. La práctica se dividirá en tres sesiones: dos sesiones de trabajo en el aula para avanzar con el diseño y otra donde se solicitará alguna modificación y se corregirá el resultado.

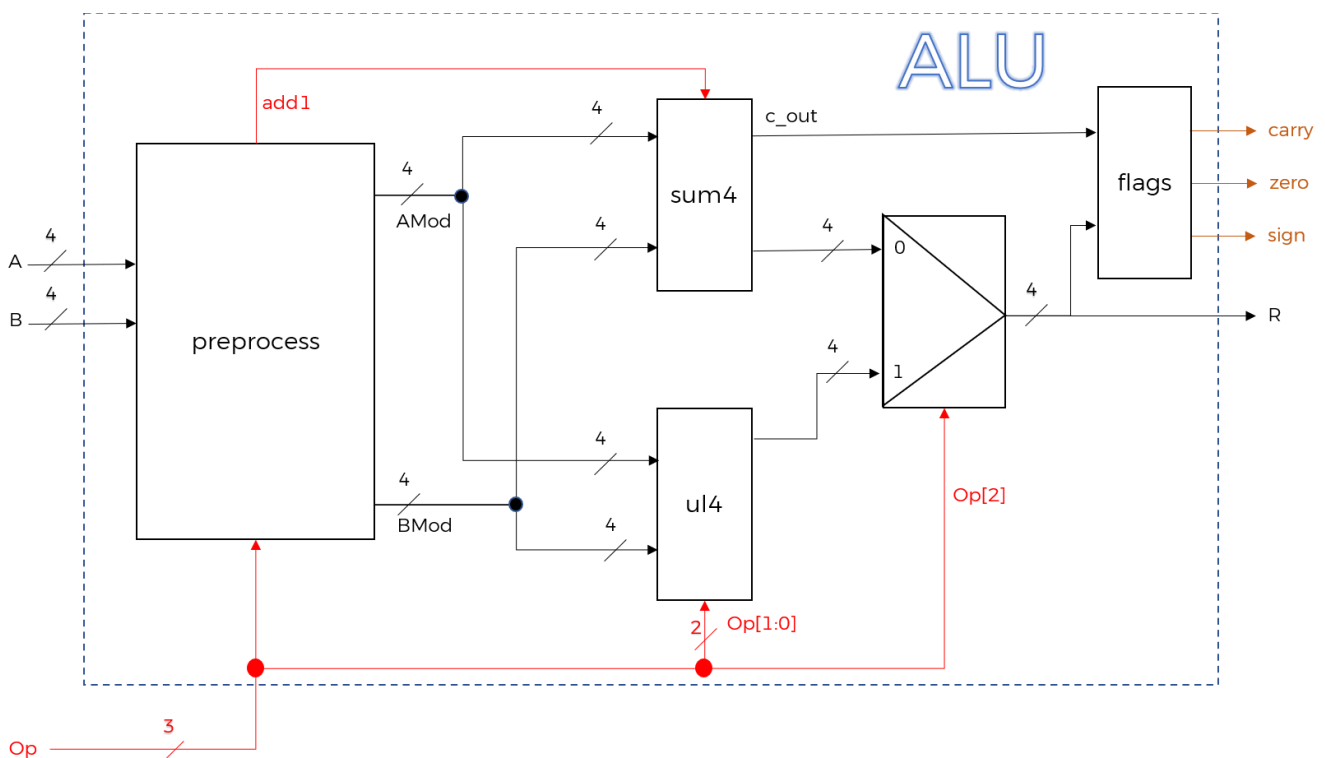
VISIÓN GENERAL

El resultado de la práctica será una pequeña ALU capaz de realizar 4 operaciones aritméticas en complemento a 2 y 4 operaciones lógicas, empleando operandos de 4 bits. La ALU tiene dos entradas de 4 bits (operandos **A** y **B**) y una salida de 4 bits (**R**). Además, es capaz de generar los *flags* asociados a las operaciones (**carry**, **zero** y **sign**). La operación a realizar se controla con la línea de 3 bits **Op**.



Op	Descripción	R
000	Identidad de A	A
001	Negativo de A	-A
010	Suma de A y B	A + B
011	Resta de A y B	A - B
100	Negación bit a bit de A	inv(A)
101	Y bit a bit entre A y B	A and B
110	O bit a bit entre A y B	A or B
111	O exclusiva bit a bit entre A y B	A xor B

Internamente, la ALU se compone de cinco grandes bloques: un sumador y una unidad lógica con operandos de 4 bits, un módulo de preprocesamiento que permite modificar el valor de los operandos **A** y **B** para poder realizar operaciones diferentes, un multiplexor que permite seleccionar entre una salida aritmética o lógica, y una unidad para calcular correctamente los flags.



Para realizar el diseño, seguiremos un proceso incremental, creando módulos para los componentes más simples que, combinados, constituirán la ALU. Escribiremos cada módulo en un fichero diferente, cuyo nombre será el mismo de cómo hayamos denominado al módulo añadiendo la extensión ".v".

PRÁCTICA 1

OBJETIVO 1. UNIDAD LÓGICA

La ALU necesita un módulo que se encargue de realizar las operaciones lógicas. Llamaremos a ese módulo la “unidad lógica”. Para construirla, debes seguir los siguientes pasos.

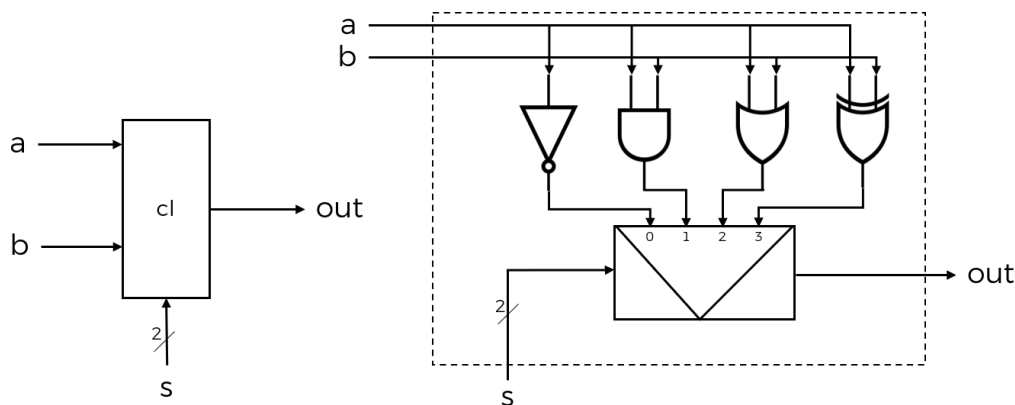
1.1. Basándonos en los ejemplos vistos en las sesiones tutorizadas, implementa un multiplexor de 4 entradas y una salida de un bit con el siguiente prototipo:

```
mux4_1(output reg out, input wire a, b, c, d, input wire [1:0] S);
```

1.2. Implementa una “celda lógica” con el prototipo siguiente:

```
cl(output wire out, input wire a, b, input wire [1:0] S);
```

Dicha celda lógica calculará sobre los bits **a** y **b** las operaciones lógicas inversión del bit **a**, and, or y xor cuando el vector de dos bits **S** vale 00, 01, 10 y 11 respectivamente.



1.3. Implementa una “unidad lógica de 4 bits” de forma estructural, utilizando las celdas lógicas descritas en el objetivo 1.2. El prototipo sería el siguiente:

```
ul4(output wire [3:0] Out, input wire [3:0] A, input wire [3:0] B, input wire [1:0] S);
```

OBJETIVO 2. SUMADOR DE 4 BITS

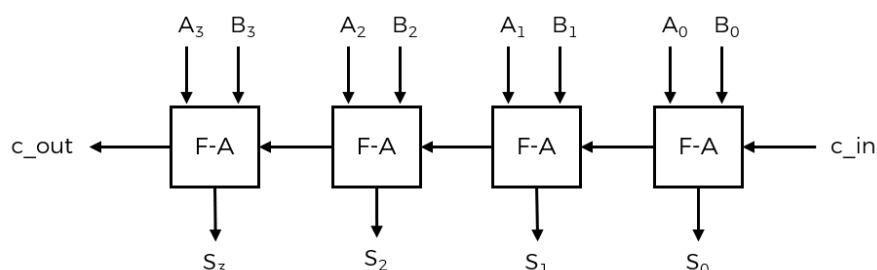
La base para realizar las operaciones aritméticas es un circuito sumador de 4 bits.

2.1. Implementa un “Full-Adder” usando asignación continua y el operador concatenación ‘{...}’, con el prototipo

```
fa(output wire c_out, sum, input wire a, b, c_in);
```

2.2. Implementa un sumador completo de 4 bits de forma estructural utilizando los “Full-Adder” descritos en el objetivo anterior. El prototipo sería el siguiente:

```
sum4(output wire [3:0] S, output wire c_out, input wire [3:0] A, input wire [3:0] B, input wire c_in);
```



PRÁCTICA 1

OBJETIVO 3. MULTIPLEXOR DE 2 ENTRADAS DE 4 BITS Y FLAGS

3.1. Basándonos en los ejemplos vistos en las sesiones tutorizadas, implementa un multiplexor de 2 entradas a una salida en el que las entradas son de 4 bits. Como ya sabes, la línea de selección **s** determina cuál de las entradas pasa a la salida. El prototipo del módulo es el siguiente:

```
mux2_4(output wire [3:0] Out, input wire [3:0] A, input wire [3:0] B, input wire s);
```

3.2. Diseña el módulo que se encarga de calcular los flags de la ALU: acarreo (**carry**), cero (**zero**) y signo (**sign**):

- **zero** tiene valor de 1 si el resultado fue 0 ($R == 0$)
- **sign** tomará el valor del bit más significativo de R
- **carry** será 1 si hubo acarreo en la operación aritmética realizada

Los bits de acarreo y signo no importan en las operaciones lógicas, así que se pueden dejar conectados igual que en las operaciones aritméticas. Usa la siguiente cabecera:

```
flags(output wire carry, zero, sign, input wire [3:0] R, input wire c_out);
```

OBJETIVO 4. LA UNIDAD DE PREPROCESAMIENTO

Con los módulos desarrollados hasta el momento deberíamos poder construir una ALU capaz de realizar la operación aritmética de suma y todas las operaciones lógicas. Sin embargo, no seríamos capaces de realizar el resto de operaciones aritméticas.

La solución es crear un pequeño módulo de preprocesamiento de los operandos que sea capaz de realizar transformaciones en **A** y **B** antes de usarlos en el sumador.

4.1. Crea un módulo capaz de cambiar el signo de un número en complemento a 1. El módulo debe incluir una señal de control **cpl**, de manera que podamos elegir entre dejar pasar el dato sin modificar o calcular el número cambiado de signo. Elabora el módulo con el prototipo siguiente:

```
compl1(output wire [3:0] Out, input wire [3:0] Inp, input wire cpl);
```

Si $cpl = 1$, $Out = Inv(Inp)$ y si no, $Out = Inp$.

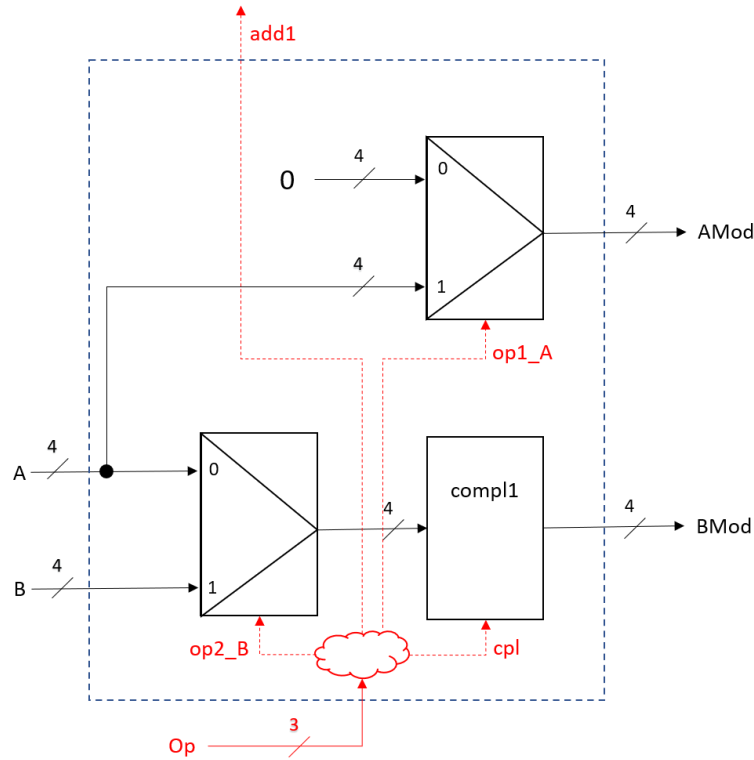
4.2. Crea el módulo de preprocesamiento de acuerdo con el siguiente prototipo

```
preprocess(output wire [3:0] AMod, output wire [3:0] BMod, output wire add1, input wire [3:0] A, input wire [3:0] B, input wire [2:0] Op);
```

En este módulo, las salidas **AMod** y **BMod** son modificaciones de **A** y **B** cuyo valor final depende de las entradas de control a cada multiplexor y al negador en complemento a 1. La nube en rojo representa la lógica que hay que implementar para, a partir de la señal general de control de la ALU (**Op**), calcular las señales de control específicas que permiten modificar los operandos. Tres de esas señales sólo se usan internamente en el módulo, pero **add1** es una salida que se conectará a la entrada de c_in del sumador que diseñaste, de manera que puedas sumar una unidad adicional.

En el caso de que vayamos a realizar una operación lógica, debemos configurar las señales de control (**op1_A**, **op2_B**, **cpl**) para que **AMod** = **A** y **BMod** = **B**. Para las operaciones aritméticas que requieren la negación de uno de sus operandos, necesitamos calcular ese número negativo en complemento a 2. Para ello, calcularemos el negativo en complemento a 1 (**cpl** = 1) y nos aseguraremos de que se sume una unidad adicional (**add1** = 1).

PRÁCTICA 1



Para crear las funciones lógicas que determinan el valor de las señales de control (**add1**, **op1_A**, **op2_B**, **cpl**) a partir del valor de **Op**, lo más recomendable es escribir las tablas de verdad de cada una y construir mediante puertas o expresiones lógicas estas funciones.

OBJETIVO 5. PONIENDO TODO JUNTO

5.1. Une todos los componentes como aparecía en la figura de la primera página de acuerdo con el siguiente prototipo:

```
alu(output wire [3:0] R, output wire zero, carry, sign, input wire [3:0] A, B, input wire [2:0] Op);
```

5.2. Diseña un testbench que te permita chequear una operación cualquiera realizada con esta alu. Este testbench servirá de complemento al que te pasaremos y te ayudará en la depuración de tu práctica.

OBJETIVO 6. PRUEBAS Y MODIFICACIÓN

Se facilitará un testbench completo para que puedan comprobar el funcionamiento de su práctica antes de la entrega (complementario al que hayan diseñado en el Objetivo 5).

En la sesión de evaluación, llevarán el diseño que hayan realizado y se les pedirá una ampliación o modificación de la práctica. Para su realización, podrán consultar los materiales disponibles en el aula virtual pero no podrán apoyarse en herramientas de IA. Ténganlo en cuenta para preparar adecuadamente su trabajo.

PRÁCTICA 1**ENTREGA Y EVALUACIÓN**

Una vez que el/la profesor/a les haya corregido la práctica e indicado su calificación, se subirá a la tarea del campus virtual un fichero comprimido que incluya todos los ficheros creados hasta ese momento.

La práctica se evaluará de 0 a 10, con el criterio que describe la siguiente rúbrica:

Nota numérica	¿Cómo se consigue?
0-4	- Presentando una práctica copiada o desarrollada con IA (nota: 0). - Presentando únicamente la parte básica de la práctica, sin ser capaces de realizar la modificación. - Si no responden satisfactoriamente a las preguntas del profesor (aunque la práctica funcione).
5-6	Si la modificación no funciona del todo, pero: - Han sabido responder adecuadamente a las preguntas del profesor
7-10	- Si la parte básica de la práctica funciona. - Si implementan la modificación con éxito. - Si responden correctamente a las preguntas del profesor La calificación final dependerá de la calidad de la implementación y las respuestas